



LISTA DE EXERCÍCIOS 5

GABARITO...

OBS. Resoluções em sala de aula podem englobar mais pontos de vista e atributos do que apenas as respostas simplificadas apresentadas no gabarito.

1. Foi comentado que uma possibilidade para lidar com problemas de seção crítica seria desabilitar as interrupções do sistema. Entretanto, efetuar esse procedimento com frequência poderia prejudicar o relógio do sistema. Explique por que isso acontece e como esses efeitos poderiam ser minimizados.

Resposta: O relógio do sistema é atualizado a cada interrupção de clock. Se as interrupções forem desabilitadas – particularmente por um longo período de tempo – é possível que o relógio do sistema perca o tempo correto. O relógio do sistema também é usado para escalonamento. Por exemplo, o quantum para um processo é expresso como um número de pulsos de relógio (*clock ticks*). A cada interrupção de relógio, o escalonador determina se o quantum para o processo em execução no momento expirou. Se as interrupções de *clock* fossem desabilitadas, o escalonador poderia atribuir de maneira não precisa os quanta. Este efeito pode ser minimizado desabilitando interrupções de *clock somente* por períodos de tempo muito curtos.

2. Forneça razões do por que SOs implementam múltiplos mecanismos de inter-travamento (*locks* – ex. *Windows, Solaris, Linux etc*). Descreva as circunstâncias sob as quais são usados *spinlocks*, semáforos, *mutexes* adaptativos e variáveis de condição. Em cada caso, explique por que o mecanismo é necessário.

Resposta: SOs oferecem diferentes mecanismos de inter-travamento dependendo das necessidades dos desenvolvedores de aplicação. *Spinlocks* são úteis para sistemas multiprocessados onde uma *thread* executa em um laço de espera ocupada (*busy-loop* – por um curto intervalo de tempo), ao invés de incorrer na sobrecarga de ser colocada em uma fila de pronto (*sleep*). *Mutexes* são úteis para o travamento de recursos. No Solaris, *mutexes* adaptativos são usados, significando que o *mutex* é implementado com um *spinlock* em máquinas multiprocessadas. Semáforos e variáveis de condição são ferramentas mais apropriadas para sincronização quando um recurso deve ser mantido por um longo período de tempo, considerando que espera ocupada (“*spinning*”) é ineficiente por um longo período de tempo.

3. Explique o conceito de atomicidade de transação.

Resposta: Uma transação pode ser vista como uma série de operações de leitura (*read*) e escrita (*write*) até que algum dado seja seguido por uma operação de *commit*. Se a série de operações em uma transação não pode ser completada, a transação deve ser abortada e as operações que ocorreram devem ser revertidas (*rolled back*). É importante que essa série de operações em uma transação apareça como uma operação indivisível para assegurar a integridade dos dados sendo atualizados. De outra forma, os dados poderiam ser

comprometidos se operações de duas (ou mais) transações diferentes forem sobrepostas (*intermixed*).

4. Qual é o significado do termo “espera ocupada” (*busy wating*)? Que outros tipos de espera existem em um SO? A espera ocupada pode ser evitada? Explique.

Resposta: Espera ocupada (*busy wating*) significa que um processo está esperando que uma condição seja satisfeita em um laço fechado sem liberar o processador. Por outro lado, um processo poderia esperar liberando o processador, e bloqueando em uma condição e esperar ser acordado em algum momento apropriado no futuro. Espera ocupada pode ser evitada, mas incorre na sobrecarga associada com a colocação do processo no estado *sleep* e acordando-o (*wake up*) quando o estado apropriado do programa é alcançado.

5. Explique por que *spinlocks* não são apropriados para sistemas uniprocessados, e ainda assim, são frequentemente usados em sistemas multiprocessados.

Resposta: *Spinlocks* não são apropriados para sistemas monoprocessados, pois a condição que tiraria o processo do laço pode ser obtida somente pela execução de um processo diferente. Caso o processo não libere o processador, outros processos não conseguem a oportunidade de ajustar a condição do programa necessária para que o primeiro processo continue. Em um sistema multiprocessado, outros processos executam em outros processadores e assim, modificam o estado do programa de modo a liberar o primeiro processo do laço (i.e. *spinlock*).

6. $\langle \rangle$ Mostre que se as operações sobre semáforo *wait()* e *signal()* não são executadas atomicamente, então a exclusão mútua é violada.

Resposta: Uma operação *wait()* decrementa atomicamente o valor associado com um semáforo. Se duas operações *wait()* são executadas em um semáforo quando o seu valor for um, se as duas operações não são realizadas atomicamente, então é possível que ambas as operações devam proceder ao decremento do valor do semáforo, violando a condição de exclusão mútua.

7. Ilustre como um semáforo binário pode ser usado para implementar exclusão mútua entre n processos.

Resposta: Os n processos compartilham um semáforo, *mutex*, inicializado em 1. cada processo P_i é organizado como segue:

```
do {  
    wait(mutex);  
    /* seção crítica */  
    signal(mutex);  
    /* restante da seção */  
} while (true);
```

8. Contextualize a questão de inversão de prioridade. Para tanto use como base a discussão sobre o problema de inversão de prioridade ocorrido no projeto Mars Pathfinder. Referência: <https://users.cs.duke.edu/~carla/mars.html>

Resposta:

A questão refere-se a verificação cíclica entre as tarefas *bc_sched* e *bc_dist*, as quais verificam a cada ciclo de execução entre elas, que a outra tarefa terminou. A tarefa *bc_sched* é a tarefa de maior prioridade no sistema (exceto pela tarefa *tExec* no SO *vxWorks*). A tarefa *bc_dist* é a terceira tarefa de prioridade maior (a tarefa que controla a entrada na atmosfera e o pouso é a segunda). Todas as tarefas que desempenham outras funções são

mais baixas. Funções científicas, como o processamento e compressão de imagens e a tarefa ASI/MET possuem prioridades ainda mais baixas. A falha a qual esta questão se refere foi identificada pela aeronave como uma falha da tarefa `bc_dist`, a qual não conseguia completar sua execução antes do início da tarefa `bc_sched`. O comportamento da aeronave ao detectar esta falha foi resetar o computador. Este reset reinicia todo o hardware e software. A aeronave também terminava a execução das atividades de solo correntes. Dados de atividades científicas ou de engenharia não são perdidos (os dados presentes na RAM são preservados desde que a energia não caia). Entretanto, as atividades previstas para o restante do dia eram completadas apenas no dia seguinte. A falha detectada foi identificada como um caso de inversão de prioridade. A tarefa `bc_dist` era bloqueada pela tarefa de menor prioridade ASI/MET, a qual estava mantendo um recurso compartilhado. A tarefa ASI/MET adquiriu este recurso e então era interrompida por outras tarefas de prioridade intermediária. Quando a tarefa `bc_sched` era ativada, para ajustar as transações para o próximo ciclo do barramento 1553, ela detectava que a tarefa `bc_dist` ainda não completou sua execução. O recurso que causava este problema foi um semáforo de exclusão mútua usado dentro do mecanismo `select()` para controlar o acesso à lista de descritores de arquivo (`wait()`). O mecanismo `select()` cria um semáforo de exclusão mútua para proteger a “lista de espera” dos descritores de arquivo para aqueles dispositivos que suportam a chamada `select()`. O mecanismo `pipe()` do `vxWorks` faz uso do `select()` e o IPC usado no projeto é baseado em pipes. A tarefa ASI/MET chama o `select()`, o qual chama `pipeioctl()`, o qual, por sua vez, chama `selNodeAdd()`, que estava no processo de liberar o semáforo mutex. A tarefa ASI/MET era interrompida e a chamada `semGive()` não era completada. Com isso, várias tarefas de prioridade intermediária executavam até que a tarefa `bc_dist` fosse ativada. A tarefa `bc_dist` tentava enviar os dados mais recentes da tarefa ASI/MET pelo mecanismo de IPC, o qual chamava `pipeWrite()`. O `pipeWrite()` bloqueava, tomando o semáforo mutex. Mais tarefas de prioridade média executavam, ainda não permitindo que a tarefa ASI/MET executasse, até que `bc_sched` fosse acordada. Nesse ponto, a tarefa `bc_sched` determinava que a tarefa `bc_dist` não completou seu ciclo (deadline estrito no sistema) e declarava o erro que iniciava o reset.

9. Discuta brevemente a questão de starvation em monitores conforme apresentado em <http://vip.cs.utsa.edu/nsf/pubs/starving/starving.html>

Resposta: o problema dos filósofos é discutido em muitos livros textos e é um problema clássico ilustrando os conceitos de regiões críticas e sincronização. Cinco filósofos sentam em uma mesa circular. Na frente de cada um há um prato de arroz ou macarrão, e entre cada prato há um hashi ou garfo. Para comer, um filósofo deve pegar ambos os hashis/garfos próximos ao prato. Cada filósofo está em um dos estados: *thinking*, *hungry* ou *eating*. Quando um filósofo está pensando (*thinking*) e fica com fome, ele muda o seu estado para *hungry* e tenta obter ambos os hashis/garfos necessários. Caso consiga, ele muda seu estado para *eating* por algum tempo, e então devolve os hashis/garfos e entra novamente no estado *thinking*.

Vários livros de SO fornecem essencialmente a mesma solução de monitor para este problema. A maioria das discussões do problema do jantar dos filósofos requerem que uma solução não permita que um filósofo morra de fome (*starve*). Em Silberschatz e Galvin:

“Finalmente, quaisquer soluções satisfatórias para o problema do jantar dos filósofos devem possuir salvaguardas contra a possibilidade de um dos filósofos morrer de fome (*starve*). Uma solução livre de *deadlock* não necessariamente elimina a possibilidade de *starvation*.”

O tratamento de *starvation* em uma solução baseada em monitores depende intrinsecamente da implementação do monitor. Isso pode depender da prioridade relativa das filas de espera e de entrada e da disciplina de acesso usada nessas filas (FIFO, LIFO, aleatória etc).

Silberschatz e Galvin representam as filas como sendo FIFOs, mas não discutem explicitamente esta questão.

A maioria dos livros texto apresenta uma solução similar para o problema do jantar dos filósofos. Cada filósofo está em um dos estados *thinking*, *hungry*, ou *eating*. Quando um filósofo termina de comer, os vizinhos são verificados novamente. Um vizinho com fome (*hungry*) cujo vizinho não esteja comendo pode mudar seu estado para *eating*. Com este algoritmo, um filósofo pode morrer de fome (*starve*) se um de seus vizinhos está sempre comendo.

Em um contexto teórico, *starvation* significa geralmente um bloqueio indefinido, significando que não existe um tempo limite (ou mais precisamente, o número de vezes que outros filósofos comem) entre os estados *hungry* e *eating* para um dado filósofo. Na prática, seria interessante definir um tempo limite relativamente pequeno. Considerando que existem cinco filósofos, o limite deveria estar por volta de 5 e não 5000. O algoritmo educado (*polite algorithm*) evita *starvation*. Entretanto, paga-se um preço considerando que alguns dos filósofos permanecem com fome durante um tempo maior do que ocorreria normalmente. Outra forma de dizer isso é que a solução não permite tanto paralelismo quanto a tradicional.